Multimedia Tools and Applications manuscript No.
(will be inserted by the editor)

A mechanism for personalized Automatic Speech Recognition for less frequently spoken languages: the Greek case

Panagiotis Antoniadis · Emmanouil Tsardoulias · Andreas Symeonidis

Received: date / Accepted: date

Abstract Automatic Speech Recognition (ASR) has become increasingly popular since it significantly simplifies human-computer interaction, providing a more intuitive way of communication. Building an accurate, general-purpose ASR system is a challenging task that requires a lot of data and computing power. Especially for languages not widely spoken, such as Greek, the lack of adequately large speech datasets leads to the development of ASR systems adapted to a restricted corpus and/or for specific topics. When used in specific domains, these systems can be both accurate and fast, without the need for large datasets and extended training. An interesting application domain of such narrow-scope ASR systems is the development of personalized models that can be used for dictation. In the current work we present three personalization-via-adaptation modules, that can be integrated into any ASR/dictation system and increase its accuracy. The adaptation can be applied both on the language model (based on past text samples of the user) as well as on the acoustic model (using a set of user's narrations). To provide more precise recommendations, clustering algorithms are applied and topic-specific language models are created. Also, heterogeneous adaptation methods are combined to provide recommendations to the user. Evaluation performed on a self-created database containing 746 corpora included in messaging applications and e-mails from the same user, demonstrates that the proposed approach can achieve better results than the vanilla existing Greek models.

Keywords Automatic Speech recognition · Dictation · Personalization · Clustering

Panagiotis Antoniadis
School of Electrical and Computer Engineering, National Technical University of Athens (NTUA), Greece
E-mail: pantoniadis97@gmail.com

Emmanouil Tsardoulias
School of Electrical and Computer Engineering, Faculty of Engineering, Aristotle University of Thessaloniki (AUTh), Greece
E-mail: etsardou@ece.auth.gr

Andreas Symeonidis
School of Electrical and Computer Engineering, Faculty of Engineering, Aristotle University of Thessaloniki (AUTh), Greece
E-mail: asymeon@eng.auth.gr

1 Introduction

It is a common fact that technology has penetrated our everyday life via smart assistants, mobile phones and smart devices in general. Nonetheless, computer-human interaction introduces a bottleneck effect on the efficiency of the tasks to be performed. According to Jacob et al. [18], "the problem of human-computer interaction can be viewed as a problem of two powerful information processors (human and computer) attempting to communicate with each other via a narrow-bandwidth, highly constrained interface". This is why various human-machine communication approaches are continuously explored to introduce naturalness in the interaction process. Automatic Speech Recognition (ASR) is one of the approaches that aspire to overcome this bottleneck by enabling computers to better understand and interpret human speech.

Building an efficient, general-purpose ASR system is a difficult task. Apart from the sounds they hear, humans also observe body language and eye movement, as well as have knowledge of the interlocutor when communicating with each other to understand the meaning of the spoken words [1]. In ASR, only the sound of the words is available. Each speaker's different tone and pronunciation, along with surrounding noise or noise introduced by the sound capturing device, make the proper analysis of the incoming sound even less reliable. As a result, in order to build an accurate ASR system, many hours of narration from different speakers are needed so as to properly train the model.

As for languages not widely spoken, their limited spread restricts the existence of recorded speech data needed to build an ASR system that is accurate and general at the same time. In most cases, however, when implementing an ASR system, such "generality" is not necessary. We argue that personalized and domain-specific ASR systems can accurately understand speech deriving from a single speaker, even with a smaller training set and subsequently smaller models.

To this end, this paper investigates adaptation methods for creating a personalized dictation system for less frequently spoken languages that can be integrated into any messaging/mailing service and improve its performance. According to our experiments, the training corpus, consisting of the previously sent messages of the user and a set of user's narrations (up to only a few hundred samples), can lead to adequately accurate results. As for the language the ASR system should interpret, we chose Greek which is a low resource language in terms of training general ASR models. The proposed adaptation methods can be applied to most languages that encounter the problem of limited recorded speech data and only provide a general ASR model like in Polish [45], Romanian [24], Ukrainian [36], Slovene [43], Croatian [22] and Slovak [31] [34].

Three independent adaptation modules are proposed. The first one adapts the language model on the user's previously sent messages using linear interpolation and statistical n-gram models. The second adapts the acoustic model on a set of voice dictations of the speaker using Maximum Likelihood Linear Regression (MLLR) acoustic adaptation. These dictations can be selected from his/her previously sent messages, too. Finally, a third module performs clustering of the training corpus in semantic groups using the K-means algorithm or a variation of the Latent Dirichlet Allocation (LDA) model. Using this approach, an even more fine-grained domain-specific language model can be generated (one for each cluster). All adaptation methods assume that the ASR engine can support custom language and acoustic models, as well as a custom dictionary.

The selected ASR engine is PocketSphinx [17], a real-time, continuous speech recognition system developed by Carnegie Mellon University. PocketSphinx was chosen among its competitors because it is fast, can be locally executed and requires relatively modest computational

resources [26] [13]. Also, it is open-source and fully customizable[1], since it can be configured using a language model, a dictionary and an acoustic model, satisfying the requirements set above.

The rest of this paper is organized as follows. Section 2 presents the state-of-the-art in ASR systems focusing on the Greek ASR systems that this paper concerns. Section 3 describes the implementation of the proposed methods, commenting on their transferability to other languages. Section 4 discusses the experimental results, providing a clear view of the accuracy improvements introduced by each method. This section also includes a description of the data used during the experiments. Finally, section 5 summarizes the key aspects of our work, discusses future work and concludes the paper.

2 State of the Art

In the current section, modern ASR architectures are discussed and the current state of the art of Greek ASR systems is presented.

Developments in ASR include Hidden Markov Models (HMMs) and Neural Networks of various types, sizes and architectures. These approaches have been extensively tested, compared and combined in some cases, yielding various results depending on the corresponding application constraints [41].

There are many different neural network architectures including, but not limited to, Feed Forward Neural Networks, Recurrent Neural Networks and Radial Basis Function Networks. Each architecture has its corresponding training algorithms which vary in complexity and performance. Moreover, some training algorithms might be a better fit for a specific type of data than others. In addition to combining them with HMMs, many attempts have been made to use only neural networks for speech recognition [16], [25]. Also, transfer learning-based methods [20] and sequence-to-sequence models [7] have achieved promising performance in ASR including Listen, Attend and Spell [6], Neural Transducer [19], Monotonic Alignments [33] and Recurrent Neural Aligner [35].

HMMs constitute another standard approach for ASR. They were initially used for ASR in the 1970's and have been, since then, the basis for many large vocabulary, continuous speech recognition systems [14]. They have been utilized in numerous applications including isolated digits recognition [32], transcribing broadcast news [28], spontaneous telephone conversations [44] and many more. Today, they are used by several open-source ASR tools, including PocketSphinx, Sphinx4, Kaldi and HTK [42]. In spite of their extensive use, HMMs have the same requirements for large amounts of training data with neural networks. According to CMU Sphinx tutorials, training an acoustic model to be used with PocketSphinx or Sphinx4 requires around 10 hours of recordings for a single speaker's dictation and 50 hours of recordings of 200 speakers for many speakers' dictation. Additionally, it would require a month of training time for parameters' optimization [10]. During recent years, many applications of ASR using hybrid models of neural networks combined with HMMs have emerged. Deep Neural Networks have also been used along with HMMs for speech recognition [27], [11]. These hybrid models have shown promising results by utilizing the advantages of both models and mitigating their disadvantages [39].

As far as the Greek language is concerned, the first Greek dictation system, Logotypografos 1.0, and Greek speech corpus, Logotypografia Database, was conducted for workflow improvement in the field of journalism in 2003 and achieved 19.27% Word Error Rate (WER) by using

[1] https://github.com/cmusphinx/pocketsphinx

speaker-independent genomic HMMs [12]. A WER reduction of 0.28% was obtained on the same database by using maximum entropy language models [29] that employ stemming to cope with the very large number of distinct words. In [40] the G2P approach was used to perform automatic word recognition for robotic applications, as a solution to the absence of a Greek general model. Two years later, a general Greek model, including an acoustic model, a dictionary and a language model, was created using CMU Sphinx in [30]. Its accuracy is not as high as in more popular languages, but it enables the implementation of personalized and domain-specific Greek models.

In a dictation system, the performance of the aforementioned general Greek model can be improved by adaptation methods. In this paper, we show that the language model can be adapted to past corpora (messages and/or e-mails) of the user and the acoustic model can be adapted to a set of user's narrations. The first kind of adaptation relies on linear interpolation and statistical n-grams in modeling the messages [4], while the second uses the MLLR acoustic adaptation method [38]. Also, clustering algorithms are employed to provide more adapted suggestions, tailored to specific topics, taking advantage of the fact that the sent e-mails of a single user can usually be classified in semantic groups. As a result, the language model adaptation can be applied based on a specific semantic group and not on the overall corpus of the user, possibly leading to increase of the adaptation level. These methods can be integrated into any messaging service, increasing its accuracy without requiring full online retraining, which typically introduces large delays to the process. Furthermore, different types of language model adaptation mechanisms can be combined, in order to provide the user with more suggestions (top-2 or top-3 suggestions), that are supplementary to each other. The proposed implementations of the above modules are described in the section that follows.

3 Proposed mechanism

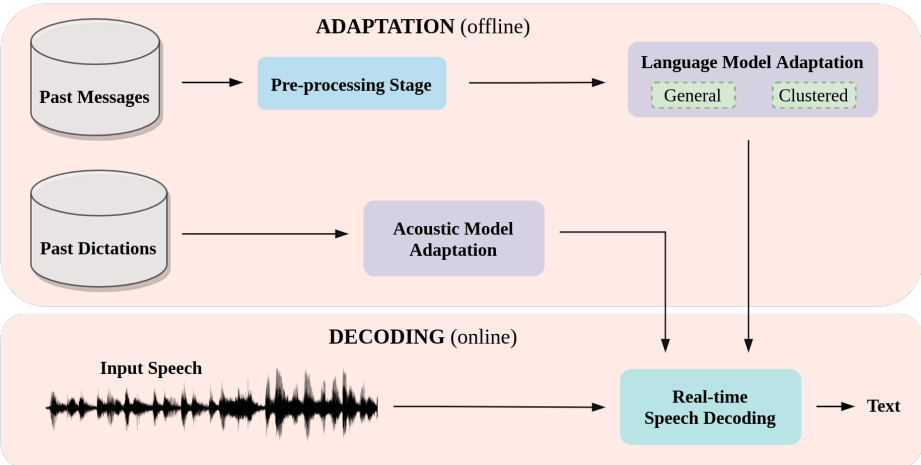


Figure 1: Overview of the proposed modules.

Our implementation comprises three distinct adaptation modules and a speech decoding module. The adaptation modules are offline, meaning that they are trained before the dictation

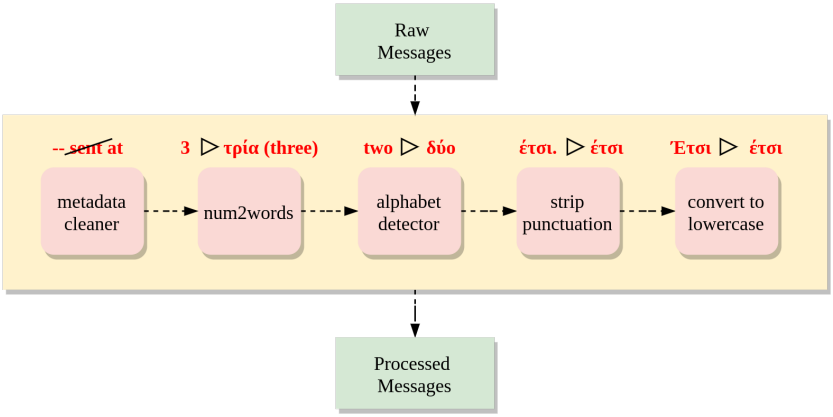


Figure 2: Pre-processing stage that converts a raw message into a sequence of alphabetical Greek words

system is used. The decoding module is responsible for the speech-to-text conversion based on the selected adaptation modules and is online, meaning that it is executed each time the user dictates a new sentence. It should be noted that our implementation relies on the existence of past corpora from the user, therefore a suitable source of information could be old e-mails or dialogues in instant messaging (IM) tools. An overview of the proposed modules is presented in Figure 1.

3.1 Pre-processing stage

Before proceeding to the adaptation modules, the raw corpora undergoes a pre-processing stage, as depicted in Figure 2. This process converts a raw string into a sequence of alphabetical Greek words, using a format that the adaptation modules can handle. First, the metadata cleaner removes any additional information provided along with the text (e.g. in the case of e-mails the subject, the sender and the receiver of the message). This step is based on a set of specialised regular expressions for each messaging tool (e.g. e-mails), since no universal format exists. Next, all numbers should be converted to text, a task that in several languages can be easily achieved with ready-to-use software (e.g. num2words[2]). In our case, we implemented a Greek version of num2words using pure Python. The next step is to translate the non-Greek words from each message using alphabet-detector[3], a package that detects in what alphabet (and not language, since several languages utilize the same alphabets) the characters of a string are written. In this step, some English words that cannot be translated to Greek are removed from the respective string. Finally, we strip punctuation and we convert all words to lowercase. Table 1 demonstrates how a raw Greek e-mail is modified during the pre-processing stage.

[2] https://pypi.org/project/num2words/
[3] https://pypi.org/project/alphabet-detector/

Step	Output in Greek	Output in English	Description
raw message	Επίσης, θέλω να δηλώσω το μάθημα Robotics με κωδικό 120. Στάλθηκε στις 27/08/2020	Also, I want to register for the Robotics course with code 120. Sent in 27/08/2020	
metadata cleaner	Επίσης, θέλω να δηλώσω το μάθημα Robotics με κωδικό 120.	Also, I want to register for the Robotics course with code 120.	The reported date at the bottom of the message is removed.
num2words	Επίσης, θέλω να δηλώσω το μάθημα Robotics με κωδικό εκατόν είκοσι.	Also, I want to register for the Robotics course with code one hundred twenty.	The number 120 is converted into the Greek word sequence 'εκατόν είκοσι'.
alphabet detector	Επίσης, θέλω να δηλώσω το μάθημα Ρομποτική με κωδικό εκατόν είκοσι.	Also, I want to register for the Robotics course with code one hundred twenty.	The English word 'Robotics' is translated to the Greek 'Ρομποτική'.
strip punctuation	Επίσης θέλω να δηλώσω το μάθημα Ρομποτική με κωδικό εκατόν είκοσι	Also I want to register for the Robotics course with code one hundred twenty	The period at the end of the sentence and the comma are removed.
convert to lowercase	επίσης θέλω να δηλώσω το μάθημα ρομποτική με κωδικό εκατόν είκοσι	also I want to register for the robotics course with code one hundred twenty	Uppercase 'E' and 'P' are converted into the lowercase 'ε' and 'ρ'.

Table 1: Example of pre-processing a sample message. In the first column each step is noted. The second and the third column present the output produced in this step in Greek and English respectively and the fourth column explains the current output.

3.2 Language model adaptation

The first out of the three proposed modules implements adaptation on the language model of the dictation system, applied on past corpora of the user. The employed language model is a statistical n-gram model, i.e. a sequence of n items, which in our case are words (an n-gram with $n = 1$ is a unigram, $n = 2$ is a bigram and so on). Each n-gram is accompanied by a probability denoting its appearance probability in a sentence. Our model is derived from the past user messages, thus the probability of each n-gram is the probability of the corresponding word sequence to appear inside a future message of the same user, assuming the author will likely not change their writing style. Based on the size of the n-grams it contains, a language model has a specific order, which is defined by the maximum length of its n-grams. For example, a language model of order one, contains only unigrams, essentially being single words. A language model of order two, contains bigrams but it also contains unigrams, whereas a language model of order three contains trigrams, bigrams and unigrams.

A language model can be defined as a "next word predictor". Given a set of words $W = \{w_1, w_2, ..., w_n\}$, a corpus which contains words only from this set and a language model upon this corpus, we can calculate the probability

$$P(w_{k_i} \mid w_{k_1} w_{k_2} ... w_{k_{i-1}}) \, \forall \, i, \; k_i \in [1, \; |W|] \tag{1}$$

which denotes "the probability of word w_{k_i} appearing after the word sequence $w_{k_1} w_{k_2} ... w_{k_{i-1}}$ where each index k_i is an integer between one and the size of the set W". Even though computing such probabilities is quite useful, we are bound by the order of the language model regarding the maximum length of the word sequences we can use. For example, we cannot calculate the probability of a sequence of four words by looking at a language model of order

Algorithm 1 Probability estimation algorithm for a word sequence using a language model

```
function PROBABILITY(w_1 w_2 ... w_n)
    if probability of the given word sequence exists then
        return probability of the given word sequence
    else
        if back-off weight of w_1 w_2 ... w_{n-1} exists then
            return PROBABILITY(w_2 w_3 ... w_n) ·
            BACKOFF_WEIGHT(w_1 w_2 ... w_{n-1})
        else
            return PROBABILITY(w_2 w_3 ... w_n)
        end if
    end if
end function
```

log(p)	Word Sequence	Backoff Weight
-2.7578	under	-0.0492
-3.0627	undertook	-0.0852
-0.7457	influence of	-0.0501
-0.9478	create a common	-

Table 2: Sample trigram language model entries

three. To overcome this issue, we use what we call a back-off weight. A back-off weight is calculated for each n-gram during the creation of the language model and is used to calculate the probability of an n-gram using the probability of an (n-1)-gram. Given the probabilities and the back-off weights of the language model, the probability of a word sequence is calculated using Algorithm 1.

In order to generate a language model from the given corpus we employ the SRI Language Modeling Toolkit (SRILM) [37]. To use SRILM on a message corpus, we need to format it in a way that each line contains a single clean sentence. The final language model file is produced in ARPA format. Table 2 contains sample entries from a trigram language model in the ARPA format; the first column contains the base-10 logarithm of the probability of the corresponding word sequence while the third column includes the back-off weight, also in a base-10 logarithm format.

Usually, the words contained in past messages of the user represent a small proportion of the total words that may appear in the dictation system. As a result, when only the generated adapted language model is used, a lot of out-of-vocabulary words may appear and the overall performance of the system may degrade. In order to resolve this problem, the adapted language model is linearly interpolated with the Greek general language model to produce a merged adapted language model. More formally, if $P_{adapted}(w)$ is the probability distribution of the adapted language model and $P_{general}(w)$ is the probability distribution of the general Greek language model, then the merged adapted language model is computed as:

$$P_{merged}(w) = \lambda P_{specific}(w) + (1 - \lambda) P_{general}(w) \tag{2}$$

where λ is a hyper-parameter of the system, that defines the balance between the two models. The above workflow is illustrated in Figure 3.

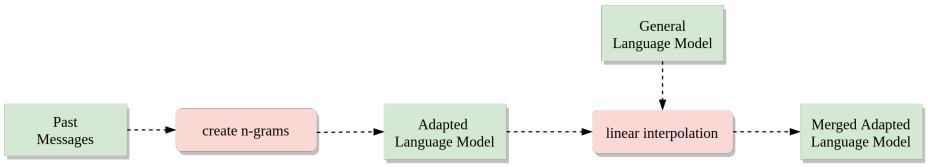


Figure 3: Language model adaptation

3.3 Acoustic model adaptation

The second module concerns the adaptation of the acoustic model with the user's tone and pronunciation. The acoustic model contains the parameters of the trained HMMs used during the ASR. PocketSphinx supports three different types of acoustic models, namely continuous, semi-continuous and phonetically tied (PTM). All three types use Gaussian Mixture Models (GMMs) and their difference is upon the number of GMMs used. A semi-continuous acoustic model uses the least GMMs out of the three (just 700) and that makes it the fastest, but the least accurate model type. A continuous acoustic model uses the most GMMs (about 150 thousand) thus being the slowest, but the most accurate model type, while a PTM lies somewhere in the middle (about 5000) [8].

The general Greek continuous acoustic model provided by CMU Sphinx, is used as the default acoustic model [30]. In order to perform acoustic adaptation to it, a set of voice dictations of a single speaker is needed along with the corresponding transcript for each recording. Each recording has a sampling rate of 16 kHz and is recorded in a single audio channel. SphinxBase[4] and SphinxTrain[5] are the two software modules employed for acoustic model adaptation. Being both part of the CMU Sphinx family, SphinxBase contains some common libraries shared between SphinxTrain and PocketSphinx, where SphinxTrain contains the scripts needed to adapt the acoustic model. The adaptation procedure is described in [9] and summarized in the next paragraph.

All the recordings (in our case WAV files) are stored in a directory along with two configuration files. The first file contains the transcripts in a format where each text is enclosed within <s> and </s> and followed by the name of the corresponding audio file inside parentheses. The second file contains the names of all the audio files to be used. For example, let an audio file named "audio1.wav" containing the recording with transcript "this is a sample recording". Then, the first configuration file should have an entry "<s> this is a sample recording </s> (audio1)" while the second configuration file an entry "audio1". After these preparatory steps are performed, we employ SphinxBase to generate the acoustic feature files. Next, statistics are extracted from the data and the final MLLR transformation is created and, finally, the transformation is applied on the general acoustic model generating the new, adapted one. The final output of the adaptation process is a new acoustic model that can be used separately and independently of the general one. The whole procedure is illustrated in Figure 4.

[4] https://github.com/cmusphinx/sphinxbase
[5] https://github.com/cmusphinx/sphinxtrain

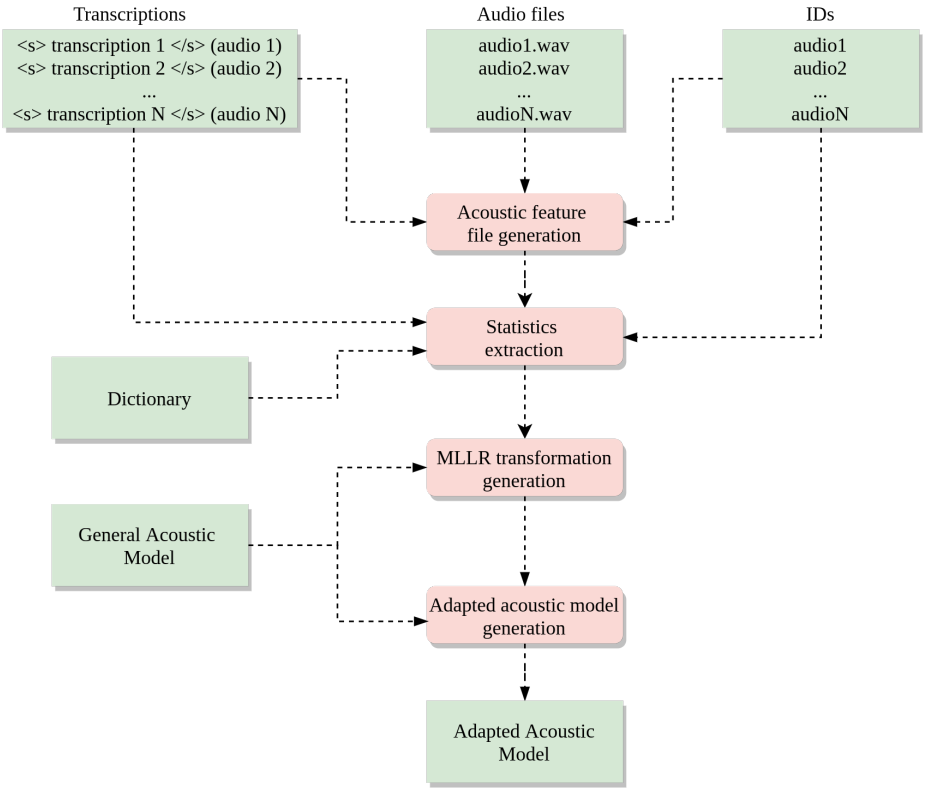


Figure 4: Acoustic model adaptation

3.4 Cluster-based language model adaptation

In order to provide more corpus-specific suggestions to the user, domain-specific language models are automatically created via clustering. Prior to clustering, the sent messages should be represented as numerical vectors. We choose to perform clustering at a sentence level and not at a message level, as large messages tend to have poor vector representation, leading to reduced overall performance of the dictation system. A brief description of the clustering algorithms follows.

3.4.1 Applying the K-means algorithm

As aforementioned, the training corpus is first converted in numerical vectors. Over the years, many types of word representations have been proposed, such as Bag of Words [21] and Word Embeddings, using cbow and skip-gram methods [23]. We choose the latter for their ability to represent rich semantic relationships between words. For the Greek language, Facebook's FastText vectors, trained on Common Crawl and Wikipedia, are available [3]. For faster vector manipulation, we use the Greek spaCy vectors[6], that have pruned the original vectors into

[6] https://spacy.io/models/el/

a vocabulary of 20,000 items, without reducing the overall accuracy. In spaCy, the vector of an entire sentence is computed as the mean value of its word vectors. After converting the sent messages into spaCy vectors, the K-means clustering algorithm is applied. This is a simple iterative algorithm that tries to partition the given data into K distinct non-overlapping subgroups (clusters). The main objective is to minimize the within-cluster sum of squares metric which is defined as:

$$J = \sum_i \sum_k w_{ik} ||x_i - \mu_k|| \tag{3}$$

where $w_{ik} = 1$ if the sample x_i belongs to cluster μ_k. In our case, the samples are defined as the vector representations of the sentences and are 300-dimensional real vectors.

3.4.2 Applying the LDA algorithm

Generally, Latent Dirichlet allocation (LDA) [2] is a generative probabilistic model for collections of discrete data such as text corpora. Each document is represented as a random mixture over latent topics and, in turn, each topic is characterized by a distribution over words. In our case, the documents correspond to the sentences and the topics correspond to the clusters. In LDA, the vector representation of a sentence is a simple word count vector. In order to convert this generative model into a clustering algorithm, we make the assumption that a sentence belongs to the topic-cluster with the highest probability.

More formally, for a set of sentences $S = \{s_1, s_2, ..., s_i\}$, LDA computes the probability distribution $P(z_k|s_i)$ over K latent topics $z_1, z_2, ..., z_K$. This distribution gives us the probability that topic z_k appears in sentence s_i. We assume that sentence s_i belongs to topic $z_{\hat{k}}$, where

$$\hat{k} = \underset{1 \leq k \leq K}{\text{argmax}}\, P(z_k|s_i) \tag{4}$$

3.4.3 Domain specific language model creation

After clustering, a language model is created for each cluster following the same procedure as in 3.2. The whole model is presented in Figure 5.

3.5 Real-time speech decoding

The online part of the dictation system takes as input a voice recording and returns as output the decoded text, requiring a dictionary, an acoustic and a language model. When clustered language adaptation is not applied, speech decoding follows the usual procedure of every ASR system. However, in clustered adaptations, in order to use the appropriate domain specific language model, the input sentence should be first assigned in one of the computed clusters. Although this procedure could be performed (or assisted) by the user, we propose an automated method (Figure 6) that is more convenient. The speech decoding module comprises of three stages. First, the input speech is converted to text by using the adapted acoustic model and the merged language model of the whole message corpus, ignoring the computed clusters. Then, the decoded text is assigned to one of the clusters. When the LDA algorithm is used, the cluster is selected in terms of the highest probability $P(z_k|s_i)$ as described in the training phase.

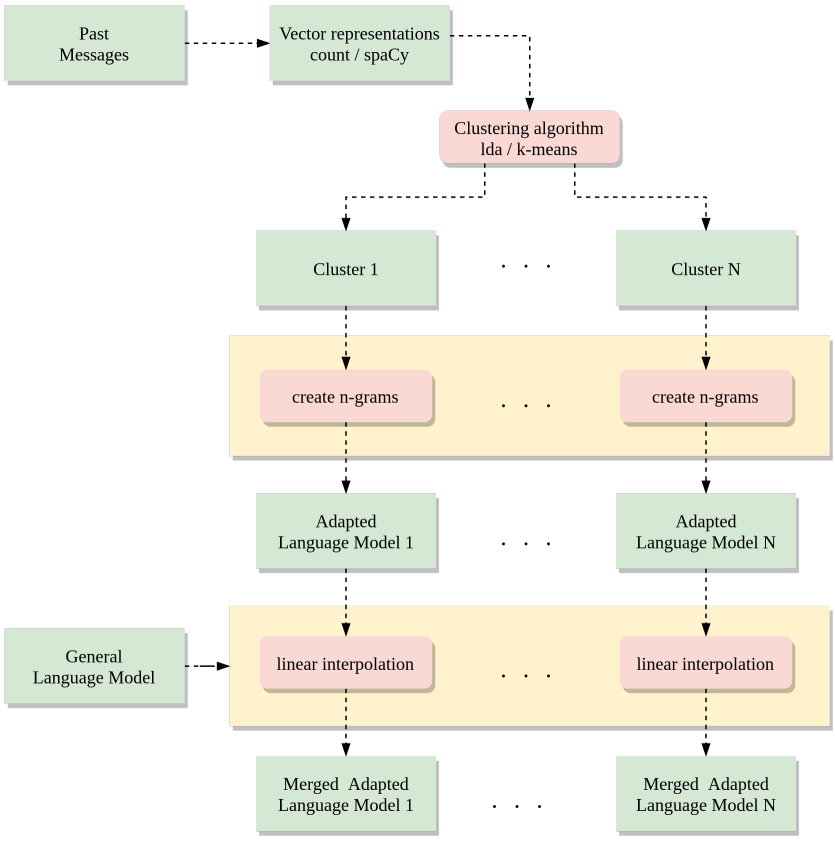


Figure 5: Clustered-based language model adaptation

When the K-means algorithm is used, the center of each cluster is available. We compute the cosine similarity of the center of each cluster with the decoded text, which is defined as:

$$\text{cosine similarity} = cos(\theta) = \frac{x\ y}{||x||||y||} \tag{5}$$

where $||x||$ is the Euclidean norm of vector x. The selected cluster is the one with the highest cosine similarity. Finally, in the third stage the input speech is converted to text again, but this time the merged adapted language model of the selected cluster is used.

3.6 Proposed method's transferability to other languages

The aforementioned proposed mechanism can easily be applied to another language with slight modifications. More specifically, in the pre-processing stage in case num2words does not support the preferred language, conversion from numbers to text should be implemented using pure Python (e.g. 5 should be replaced by the translation of the word "five" in the preferred language). In the three adaptation modules and in the speech decoding module just an acoustic model, a language model and a dictionary are required. Finally, when K-means algorithm is used, the Greek word vectors should be replaced with that of the the preferred language. In

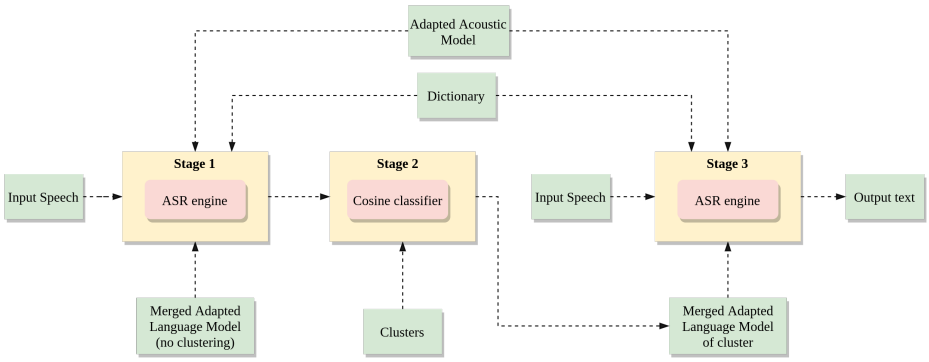


Figure 6: Real-time speech decoding after clustering

this case, FastText[7] provides vectors for 157 different languages. If the preferred language is not included in them, the LDA algorithm can be applied instead.

4 Experiments

4.1 Data aggregation

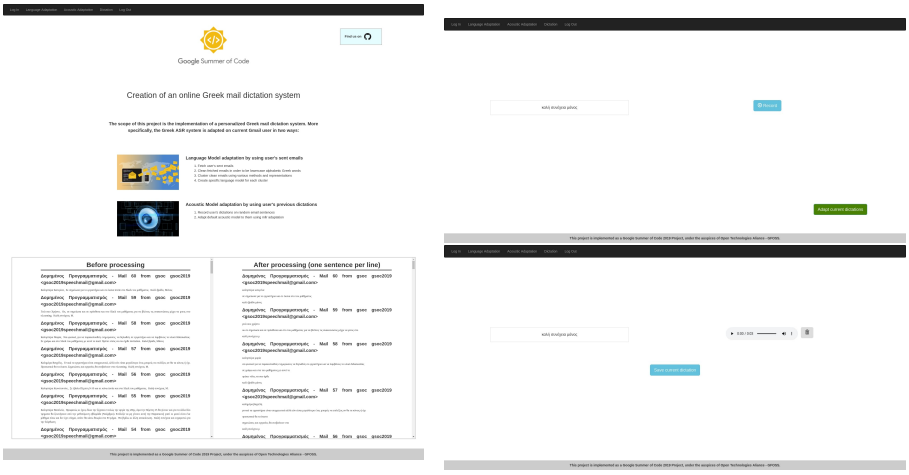
In order to evaluate the effectiveness of the proposed adaptation methods, we manually collected 746 sent IM messages of a single user. Specifically, the collected messages correspond to the responses of a male professor to his students in the mailing list of a computer science course. Then, the messages were passed through the aforementioned pre-processing stage and were dictated by the user using the built-in microphone of a common laptop without additive noise. In order to automate the recording process, we implemented fast-recorder[8], a simple Python script that is based on the sounddevice[9] package. In particular, the script takes a list of transcriptions as input and records each corresponding dictation in wav format and 16 kHz sampling rate (mono channel). In total, the collected dataset contains 746 text messages and the corresponding 746 dictations of these messages. More details about the data are presented in Table 3. As a baseline model, the Greek general acoustic and language model of CMU Sphinx are used [30], along with the provided Greek phonetic dictionary. These models have been trained using a range of different sound sources including news recordings, free audio books, audio data from academic lectures and meetings of the Greek parliament. For more information regarding the training pipeline of the Greek general models we refer readers to [30]. It should be noted that the intent of the paper is to increase the accuracy of the baseline model using the proposed adaptation modules. For that reason, any future Greek general model that is in Sphinx format can be integrated in the system, so as to achieve a better performance. Finally, our ASR system was compiled and tested on Ubuntu 18.04 LTS.

Also, in order to provide a more convenient way of collecting the data, we implemented a web user interface[10] in Angular 8. Up until now, only GMail accounts are supported by the

[7] https://fasttext.cc/docs/en/crawl-vectors.html
[8] https://github.com/PanosAntoniadis/fast-recorder
[9] https://pypi.org/project/sounddevice/
[10] https://github.com/eellak/gsoc2019-sphinx

Duration	Words	Different words	Sentences	Sampling Rate	Channels	Audio Format
77 min	9,399	2,256	746	16 kHz	1	WAV

Table 3: Dataset information



(a) Fetch and pre-process e-mails from a GMail account

(b) Dictate e-mails in the desired format

Figure 7: Web UI

web UI, but the integration of more e-mail services and IM applications is planned. Figure 7 depicts the home page of the application where the sent messages of the user are fetched from the desired e-mail service or IM application and processed in order to be in the appropriate format, while in the proceeding page the recording phase takes place. During this phase, a Greek sentence appears on the screen and the user records his/her dictation of the sentence. Then, the user can hear his/her dictation and save it for the acoustic adaptation.

4.2 Evaluation

In order to take into account both language models equally, the lambda parameter of the interpolation of language models is set to 0.5. As for the number of clusters, when a small number is selected, the clustered-based adaptation is closer to a general adaptation and the benefits of clustering diminish. On the other hand, as the number of clusters increases, the number of messages in each cluster decreases. As a result, a large number of clusters results in poor adaptation performance due to lack of enough data per cluster. Since we are not aware of the domain distribution of the messages, we cannot figure out the number of messages each cluster will contain. In case we assume that the messages are distributed uniformly among the clusters, the number of clusters can be set to:

$$k = \left\lceil \frac{dataset\ size}{messages\ per\ cluster} \right\rceil \tag{6}$$

Acoustic Model	Language Model	WER
Baseline	Baseline	35.76
Baseline	Adapted	32.79
Adapted	Baseline	32.44
Adapted	Adapted	29.41
Adapted	Adapted (K-means)	30.7
Adapted	Adapted (LDA)	30.81

Table 4: Performance comparison of proposed adaptation methods

Empirically, we observe that domain adaptation can work even with 35 to 40 messages in each cluster. So, we set $k = \lceil \frac{746}{37} \rceil = 20$ in the experiments. The evaluation metric is Word Error Rate (WER) which is defined as:

$$WER = \frac{S + D + I}{N} \tag{7}$$

where S, D and I are the number of substitutions, deletions and insertions respectively after aligning the recognized sequence with the reference sequence. N is the number of words in the reference.

In Table 4, the performance of each model is presented, after we conducted a 5-fold cross validation. In the first two columns, the type of acoustic and language model adaptation is noted and in the third column the total WER of the model is presented. We can see that the dictation system can effectively adapt to the current user. Each time an adaptation method is applied, WER decreases, meaning that the system benefits from the previously sent messages and dictations of the user.

In order to achieve an even higher decrease in WER, we should consider the following factors. First, the dataset contains a small number of messages even for a single user. Even though we utilized personalization techniques in order to reduce the amount of data an ASR system needs, the adapted language and acoustic models cannot capture the whole distribution of words and phonemes a user may dictate in such an uncontrolled environment, as an e-mail service or an IM application. This problem could be easily resolved by fetching more messages from the user's account or wait for him/her to aggregate more messages through time. Secondly, the fact that some English untranslatable words exist in some of the messages influences the total performance of the system (e.g. when slang or technical terms are used). Finally, in order to be realistic, the messages contain dates, names of places or people and other phrases that are difficult for any ASR engine to efficiently recognize efficiently. As for the clustered-based language model adaptation, the total WER is slightly higher than the general language adaptation without clustering. This happens because the domain-specific language models are not always beneficial for the system. There are cases that the input dictation may not clearly belong to a domain and should be decoded using the general adapted model. In addition, the clustering algorithms themselves put some constraints. On one hand, K-means clustering is based on spaCy vectors, that sometimes perform poorly in capturing the meaning of a sentence. On the other hand, LDA-based clustering employs count vectors and not embeddings, that are considered a very simple word representation. In case better Greek vectors are developed in the future, their integration in the system is very simple. Overall, when only one output suggestion is presented, general language adaptation is the best proposed method.

However, when a dictation system is at hand (e.g. in e-mails composition), it is desirable to provide more than one suggestion to the user and let him/her decide on the final output of the engine. Since the clustered adaptation is beneficial in some cases, we can combine it with

Method	WER
Baseline	35.76
Top-1 best method	29.41
Top-2 combining general and K-means	27.43
Top-2 combining general and LDA	27.52
Top-3 combining general, K-means and LDA	26.46

Table 5: Performance comparison using more suggestions

the general adaptation method and provide the top two suggestions to the user. In Table 5, the performance of these models is presented. We define top-2 WER as the WER of the best out of the two suggestions to the user (and top-3 WER respectively). We can see that, by providing more suggestions to the user, clustered adaptation can be used along with the general adaptation and provide an even better dictation system. Finally, when general language adaptation and clustered adaptation with K-means and LDA are combined, we achieve a 26.46% top-3 WER.

Table 6 shows a comparison of our method with other ASR systems that support the Greek language. Along with WER we present the size of each model since it is an important parameter in an off-line dictation system as in our case. Vosk [5] is a speech recognition toolkit of Alpha Cephei that supports 18 languages and dialects including Greek. While Vosk achieves better WER than the general model of CMU Sphinx, the size of the model is about ten times larger. As we can see our adaptation method achieves similar performance to Vosk keeping at the same time the size of the model small. Also, our adaptation method is dependent on the general model that uses. Therefore, it can achieve even higher performance in case a new general Greek model of Sphinx is trained. Google Speech-to-Text system [15] supports more than 125 languages and uses advanced deep learning algorithms to achieve state-of-the-art recognition performance. As expected, Google Speech-to-Text system achieves much better performance than Vosk and our system, since it is a proprietary, closed-source system that has been trained for large time spans on huge speech data sets in the cloud by state-of-the art algorithms.

Model	Size (Mb)	WER
CMU Sphinx General [30]	113	35.76
Alpha Cephei Vosk [5]	1,100	28.58
Google Cloud Speech-to-Text [15]	-	11.26
CMU Sphinx Adapted (ours)	114	29.41

Table 6: Performance comparison with other ASR systems that support Greek

5 Conclusions and Future Work

In this work, three adaptation methods are proposed that can enhance the performance of any dictation system. Specifically, the language model is adapted to the previously sent messages of the user and the acoustic model is adapted to the user's previous dictations. Also, two clustering algorithms are applied, K-means and a variation of LDA, in order to create domain-specific language models and provide more adapted suggestions. Experimental results on a self-created database have demonstrated that these adaptation methods can improve the performance of the

baseline Greek model and provide more accurate suggestions. Meanwhile, the combination of clustered and general language adaptation can provide more than one final suggestions to the user and surpass the weaknesses of each method.

In future work, we intend to explore the role of the phonetic dictionary in a personalized dictation system. We think that the mapping from words to phonemes that exists in a dictionary can be adapted to the current user given a set of his dictations and a proper adaptation algorithm. In addition, in order to improve the clustered-based adaptation, a variation of lda2vec algorithm sounds an interesting idea since it will combine the advantages of LDA topic modelling with a more powerful word representation than the count vectors. Regarding the web UI, we plan to integrate more e-mail providers and IM applications, in order to facilitate the tedious procedure of collecting speech data for adaptation. Also, a web app that can be included in a mailing client and adapt the user's dictations may be implemented in the future.

Funding

Part of this work was supported by Google Summer of Code as an open source project [11].

Conflict of interest

The authors declare that they have no conflict of interest.

Code availability

The code is available on GitHub[12].

References

1. Arora SJ, Singh RP (2012) Automatic speech recognition: a review. International Journal of Computer Applications 60(9)
2. Blei DM, Ng AY, Jordan MI (2003) Latent dirichlet allocation. Machine Learning Research 3:993–1022, DOI 10.1162/jmlr.2003.3.4-5.993
3. Bojanowski P, Grave E, Joulin A, Mikolov T (2017) Enriching word vectors with subword information. Transactions of the Association for Computational Linguistics 5:135–146, DOI 10.1162/tacl_a_00051
4. Brown PF, Della Pietra VJ, Desouza PV, Lai JC, Mercer RL (1992) Class-based n-gram models of natural language. Computational Linguistics 18(4):467–480
5. Cephei A (accessed May 21, 2021) Vosk offline speech recognition API. URL `https://alphacephei.com/vosk/`
6. Chan W, Jaitly N, Le Q, Vinyals O (2016) Listen, attend and spell: A neural network for large vocabulary conversational speech recognition. In: Proc. IEEE ICASSP, pp 4960–4964

[11] https://summerofcode.withgoogle.com/archive/2019/projects/4683496042266624/

[12] https://github.com/PanosAntoniadis/personalized_asr

7. Chiu CC, Sainath TN, Wu Y, Prabhavalkar R, Nguyen P, Chen Z, Kannan A, Weiss RJ, Rao K, Gonina E, et al. (2018) State-of-the-art speech recognition with sequence-to-sequence models. In: Proc. IEEE ICASSP, pp 4774–4778
8. CMU (accessed May 21, 2021) Sphinx: Acoustic Model Types. URL https://cmusphinx.github.io/wiki/acousticmodeltypes/
9. CMU (accessed May 21, 2021) Sphinx: Adapting the default acoustic model. URL https://cmusphinx.github.io/wiki/tutorialadapt/
10. CMU (accessed May 21, 2021) Sphinx: Training Acoustic Model. URL https://cmusphinx.github.io/wiki/tutorial/
11. Dahl GE, Yu D, Deng L, Acero A (2011) Large vocabulary continuous speech recognition with context-dependent dbn-hmms. In: Proc. IEEE ICASSP, pp 4688–4691, DOI 10.1109/icassp.2011.5947401
12. Digalakis V, Oikonomidis D, Pratsolis D, Tsourakis N, Vosnidis C, Chatzichrisafis N, Diakoloukas V (2003) Large vocabulary continuous speech recognition in greek: Corpus and an automatic dictation system. In: Proc. EUROSPEECH, pp 1565–1568
13. Gaida C, Lange P, Petrick R, Proba P, Malatawy A, Suendermann-Oeft D (2014) Comparing open-source speech recognition toolkits. Tech Rep, DHBW Stuttgart
14. Gales M, Young S (2008) Application of hidden Markov models in speech recognition. Now Foundations and Trends, DOI 10.1561/2000000004
15. Google (accessed May 21, 2021) Cloud Speech-to-Text API. URL https://cloud.google.com/speech-to-text
16. Graves A, Mohamed Ar, Hinton G (2013) Speech recognition with deep recurrent neural networks. In: Proc. IEEE ICASSP, pp 6645–6649, DOI 10.1109/icassp.2013.6638947
17. Huggins-Daines D, Kumar M, Chan A, Black AW, Ravishankar M, Rudnicky AI (2006) Pocketsphinx: A free, real-time continuous speech recognition system for hand-held devices. In: Proc. IEEE ICASSP, vol 1, pp I–I, DOI 10.1109/icassp.2006.1659988
18. Jacob RJ, Leggett JJ, Myers BA, Pausch R (1993) Interaction styles and input/output devices. Behaviour and Information Technology 12(2):69–79, DOI 10.1080/01449299308924369
19. Jaitly N, Le QV, Vinyals O, Sutskever I, Sussillo D, Bengio S (2016) An online sequence-to-sequence model using partial conditioning. In: Proc. Advances in NIPS, Curran Associates, Inc., vol 29
20. Kunze J, Kirsch L, Kurenkov I, Krug A, Johannsmeier J, Stober S (2017) Transfer learning for speech recognition on a budget. In: Proc. RepL4NLP, pp 168–177
21. Macskassy SA, Hirsh H, Banerjee A, Dayanik AA (2003) Converting numerical classification into text classification. Artificial Intelligence 143(1):51–77, DOI 10.1016/s0004-3702(02)00359-4
22. Martinčić-Ipšić S, Pobar M, Ipšić I (2011) Croatian large vocabulary automatic speech recognition. Automatika 52(2):147–157, DOI 10.1080/00051144.2011.11828413
23. Mikolov T, Chen K, Corrado G, Dean J (2013) Efficient estimation of word representations in vector space. arXiv preprint arXiv:13013781
24. Militaru D, Gavat I, Dumitru O, Zaharia T, Segarceanu S (2009) Protologos, system for romanian language automatic speech recognition and understanding (asru). In: Proc. SpeD, pp 1–9, DOI 10.1109/sped.2009.5156171
25. Mohamed Ar, Yu D, Deng L (2010) Investigation of full-sequence training of deep belief networks for speech recognition. In: Proc. INTERSPEECH, pp 2846–2849
26. Morbini F, Audhkhasi K, Sagae K, Artstein R, Can D, Georgiou P, Narayanan S, Leuski A, Traum D (2013) Which asr should i choose for my dialogue system? In: Proc. SIGDIAL, pp 394–403

27. Morgan N, Bourlard H (1990) Continuous speech recognition using multilayer perceptrons with hidden markov models. In: Proc. IEEE ICASSP, pp 413–416, DOI 10.1109/icassp.1990.115720
28. Mulbregt Pv, Carp I, Gillick L, Lowe S, Yamron J (1998) Text segmentation and topic tracking on broadcast news via a hidden markov model approach. In: Proc. ICSLP, pp 2519–2522
29. Oikonomidis D, Digalakis V (2003) Stem-based maximum entropy language models for inflectional languages. In: Proc. EUROSPEECH, pp 2285–2288
30. Pantazoglou F, Papadakis N, Kladis G (2017) Implementation of the generic greek model for cmu sphinx speech recognition toolkit. In: Proc. eRA-12
31. Pleva M, Juhár J (2014) Tuke-bnews-sk: Slovak broadcast news corpus construction and evaluation. In: Proc. LREC, pp 1709–1713
32. Rabiner LR, Juang BH, Levinson SE, Sondhi MM (1985) Recognition of isolated digits using hidden markov models with continuous mixture densities. AT&T Technical Journal 64(6):1211–1234, DOI 10.1002/j.1538-7305.1985.tb00272.x
33. Raffel C, Luong MT, Liu PJ, Weiss RJ, Eck D (2017) Online and linear-time attention by enforcing monotonic alignments. In: Proc. ICML, pp 2837–2846
34. Rusko M, Juhár J, Trnka M, Staš J, Darjaa S, Hládek D, Sabo R, Pleva M, Ritomskỳ M, Lojka M (2014) Slovak automatic dictation system for judicial domain. In: Proc. Human Language Technology Challenges for Computer Science and Linguistics, pp 16–27
35. Sak H, Shannon M, Rao K, Beaufays F (2017) Recurrent neural aligner: An encoder-decoder neural network model for sequence to sequence mapping. In: Proc. INTERSPEECH, vol 8, pp 1298–1302
36. Schlippe T, Volovyk M, Yurchenko K, Schultz T (2013) Rapid bootstrapping of a ukrainian large vocabulary continuous speech recognition system. In: Proc. IEEE ICASSP, pp 7329–7333, DOI 10.1109/icassp.2013.6639086
37. Stolcke A (2002) Srilm-an extensible language modeling toolkit. In: Proc. ICSLP, pp 901–904
38. Tamura M, Masuko T, Tokuda K, Kobayashi T (2001) Adaptation of pitch and spectrum for hmm-based speech synthesis using mllr. In: Proc. IEEE ICASSP, pp 805–808 vol.2, DOI 10.1109/ICASSP.2001.941037
39. Trentin E, Gori M (2001) A survey of hybrid ann/hmm models for automatic speech recognition. Neurocomputing 37(1-4):91–126, DOI 10.1016/s0925-2312(00)00308-8
40. Tsardoulias EG, Symeonidis AL, Mitkas PA (2015) An automatic speech detection architecture for social robot oral interaction. In: Proc. Audio Mostly 2015 on Interaction With Sound, DOI 10.1145/2814895.2814931
41. Waibel A, Hanazawa T, Hinton G, Shikano K, Lang K (1988) Phoneme recognition: neural networks vs. hidden markov models. In: Proc. IEEE ICASSP, pp 107–108, DOI 10.1109/icassp.1988.196523
42. Young S, Evermann G, Gales M, Hain T, Kershaw D, Liu X, Moore G, Odell J, Ollason D, Povey D, et al. (2002) The htk book. Cambridge university 3(175):12
43. Zgank A, Vitez AZ, Verdonik D (2014) The slovene bnsi broadcast news database and reference speech corpus gos: Towards the uniform guidelines for future work. In: Proc. LREC, pp 2644–2647
44. Zhang L, Renals S (2006) Phone recognition analysis for trajectory hmm. In: Proc. INTERSPEECH, pp 589–592
45. Ziolko B, Jadczyk T, Skurzok D, Żelasko P, Gałka J, Pędzimąż T, Gawlik I, Pałka S (2015) Sarmata 2.0 automatic polish language speech recognition system. In: Proc. ISCA, pp 1062–1063